

Practical and Architectural Constraints

Despite its advantages, full Lean integration was not feasible within the scope and constraints of this undergraduate project.

The primary limitation was architectural complexity. Integrating Lean would require designing and maintaining a backend service capable of:

- Receiving user proof submissions
- Translating structured proof steps into syntactically valid Lean code
- Executing Lean scripts securely
- Returning structured validation responses to the frontend

This introduces non-trivial engineering challenges, particularly around:

- Asynchronous communication
- Error parsing and interpretation
- Session management
- Server resource allocation

A second major constraint is secure execution and sandboxing. Allowing users to submit dynamically generated Lean scripts requires careful isolation to prevent arbitrary code execution risks. Implementing a secure sandboxing environment significantly increases deployment overhead and infrastructure complexity.

Third, translation between a natural deduction interface and Lean's proof language is non-trivial. Lean proofs are not written as direct natural deduction line-by-line derivations; instead, they often use tactic-based or term-style proofs. Mapping user-generated inference steps into valid Lean constructs would require:

- A formal mapping layer
- Additional validation logic
- Error interpretation mechanisms

This translation layer alone could constitute a substantial project.

Another practical limitation is performance and responsiveness. The implemented system prioritises instant feedback. Introducing server-side verification would create network latency and potential bottlenecks, negatively impacting user experience.

Given these boundaries, prioritising a lightweight local validation engine ensured:

- Feasibility
- Stability
- Usability
- Deliverability within timeframe

Thus, while Lean remains a technically superior verification mechanism, implementing it would have shifted the project focus from educational tool design toward distributed systems engineering and formal prover integration — exceeding the intended scope.